ĐẠI HỌC KINH TẾ QUỐC DÂN
TRƯỜNG CÔNG NGHỆ
KHOA KHOA HỌC DỮ LIỆU VÀ TRÍ TUỆ NHÂN TẠO

NHẬP MÔN CƠ SỞ DỮ LIỆU

BÁO CÁO

Đề tài: Quản lý mượn trả sách thư viện

Đề 05 – Cơ sở dữ liệu MySQL: library_mini

Giảng viên hướng dẫn: TS.Trần Đức Minh

Sinh viên thực hiện: Phan Thanh Thảo_11257758

Nguyễn Gia Hưng_11253367

Lê Hải Yến_11259127

Lớp: Trí tuệ nhân tạo 67A (AI67A)

Link Repository:
https://github.com/ghbigbrain/midterm_group1_introduction_to_database

Hà Nội, tháng 11 năm 2026

Mục lục

1. Giới thiệu **2**
2. Phân tích yêu cầu và mô hình thực thể – liên kết **2**
3. Thiết kế lược đồ (schema.sql) **4**
4. Dữ liệu mẫu (seed.sql) **6**
5. Truy vấn dữ liệu (queries.sql) **10**
Câu 1. Lượt mượn Completed trong tháng 01/2025 10
Câu 2. Số lượt đăng ký mượn theo thể loại trong quý I/2025 11
Câu 3. Bạn đọc có lượt mượn chưa có biên bản trả sách 11
Câu 4. Phí phát sinh trung bình trên mỗi giao dịch Completed theo thể loại 12
Câu 5. Ba bản sách có nhiều lượt mượn Completed nhất năm 2025 13
Câu 6. Tổng tiền từng bạn đọc đã thanh toán trong năm 2025 13
Câu 7. Bạn đọc chưa có bản ghi thanh toán nào 14
Câu 8. Tổng phí phát sinh đã xác nhận của toàn bộ bạn đọc 15
Câu 9. Tỷ lệ hủy của từng bản sách trong năm 2025 16
Câu 10. Bạn đọc có hơn 3 lượt mượn Completed trong năm 2025 16
6. Tạo VIEW (views.sql) **17**
v_detail: mỗi dòng một lượt mượn 17
v_group_summary: mỗi dòng một thể loại 18
v_debt: mỗi dòng một bạn đọc 18
7. Đánh giá và hạn chế **19**
8. Kết luận **20**

1. Giới thiệu

Bài thực hành xây dựng CSDL MySQL `library_mini` cho một thư viện, gồm lược đồ 6 bảng, 4 trigger, dữ liệu mẫu, 10 truy vấn và 3 VIEW. Bốn tệp nộp (schema.sql, seed.sql, queries.sql, views.sql) chạy lần lượt trên CSDL mới tạo và đáp ứng đủ yêu cầu của đề.

Mục tiêu. Thực hành DDL, DML, JOIN, GROUP BY, HAVING; xây dựng VIEW chi tiết, VIEW thống kê theo thể loại sách và VIEW theo dõi công nợ.

Bối cảnh. Thư viện quản lý bạn đọc, bản sách và từng lượt đăng ký mượn. Mỗi lượt mượn gồm một bạn đọc và một bản sách, có `due_date` không trước `loan_date`. Phí phát sinh do quá hạn hoặc hư hỏng được nhập theo biên bản trả sách; hệ thống không tự tính tiền phạt và không kiểm tra các khoảng mượn trùng nhau.

Quy định chung của đề được áp dụng

- Khai báo PRIMARY KEY và FOREIGN KEY; tiền dùng DECIMAL, thời điểm dùng DATE.
- NOT NULL cho thông tin bắt buộc; UNIQUE, CHECK, ENUM khi phù hợp.
- Các kiểm tra giữa nhiều bảng có thể viết thành truy vấn trong seed.sql; đề không bắt buộc trigger hay stored procedure. Không dùng CHECK để truy vấn bảng khác. Bài làm vẫn bổ sung trigger để chặn các quy tắc nhiều bảng ngay lúc ghi dữ liệu.
- Báo cáo theo năm dùng năm 2025; quý I là 01/01/2025 đến 31/03/2025; không nêu thời gian thì xét toàn bộ dữ liệu.

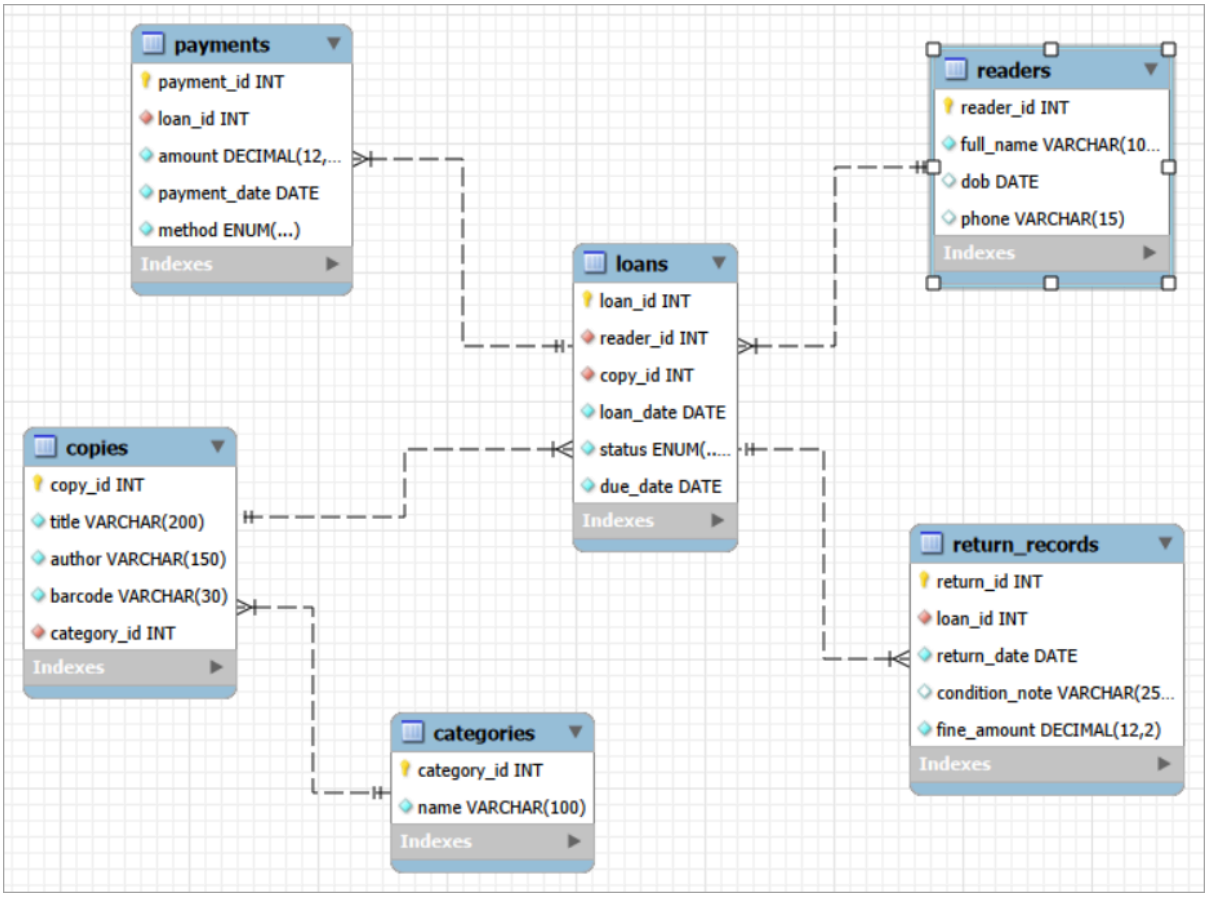
Thứ tự chạy. schema.sql → seed.sql → queries.sql → views.sql.

2. Phân tích yêu cầu và mô hình thực thể – liên kết

CSDL gồm 6 bảng nối bằng 5 khóa ngoại; bảng `loans` là trung tâm, nối bạn đọc, bản sách, biên bản trả sách và các lần thanh toán. Các quan hệ được liệt kê ở bảng dưới và thể hiện trong Hình 1.

Quan hệ	Bản số	Khóa ngoại	Ý nghĩa
categories – copies	1 – N	copies.category_id	Một thể loại có nhiều bản sách; mỗi bản sách thuộc đúng một thể loại
readers – loans	1 – N	loans.reader_id	Một bạn đọc có nhiều lượt đăng ký mượn
copies – loans	1 – N	loans.copy_id	Một bản sách được mượn nhiều lần ở các thời điểm khác nhau

Quan hệ	Bản số	Khóa ngoại	Ý nghĩa
loans – return_records	1 – 0..1	return_records.loan_id (UNIQUE)	Mỗi lượt mượn có tối đa một biên bản trả sách
loans – payments	1 – N	payments.loan_id	Một lượt mượn có thể được thanh toán nhiều lần



Hình 1. Sơ đồ quan hệ giữa 6 bảng của CSDL library_mini (MySQL Workbench)

Hai nhánh dưới `loans` khác nhau về bản số: mỗi lượt mượn có tối đa một biên bản trả (0..1) nhưng có thể có nhiều lần thanh toán (N). MySQL Workbench vẽ nhánh `return_records` với bản số "nhiều" vì không biểu diễn ràng buộc UNIQUE trên `return_records.loan_id`; ràng buộc này mới là điều bảo đảm tối đa một biên bản cho mỗi lượt mượn.

Quy tắc liên quan nhiều bảng. Các quy tắc sau liên quan đến hai bảng nên CHECK không diễn đạt được; chúng được kiểm bằng truy vấn trong seed.sql (mục 4):

- Chỉ lượt mượn `Completed` có biên bản trả; mỗi lượt `Completed` có đúng một biên bản.
- `return_date` không trước `loan_date`.
- Chỉ thu tiền khi đã có chi phí xác nhận (> 0), ngày thu không sớm hơn ngày lập biên bản; tổng tiền thu không vượt chi phí.

Dữ liệu trùng tên. Hai bản sách có thể cùng tên sách (`copies` 1 và 2, "Dế Mèn phiêu lưu ký") và hai bạn đọc có thể trùng họ tên (`readers` 1 và 6, "Nguyễn Văn An"), nên mọi kết quả truy vấn đều kèm mã định danh.

3. Thiết kế lược đồ (schema.sql)

schema.sql tạo CSDL `library_mini` (utf8mb4, utf8mb4_unicode_ci) và 6 bảng InnoDB; các lệnh DROP TABLE đặt theo thứ tự ngược phụ thuộc nên tệp chạy lại được nhiều lần. Cuối tệp là 4 trigger kiểm tra các quy tắc nhiều bảng.

Bảng	Cột	Kiểu dữ liệu	Ràng buộc
categories	category_id	INT AUTO_INCREMENT	PRIMARY KEY
categories	name	VARCHAR(100)	NOT NULL, UNIQUE
copies	copy_id	INT AUTO_INCREMENT	PRIMARY KEY
copies	title	VARCHAR(200)	NOT NULL
copies	author	VARCHAR(150)	NOT NULL
copies	barcode	VARCHAR(30)	NOT NULL, UNIQUE
copies	category_id	INT	NOT NULL, FOREIGN KEY → categories
readers	reader_id	INT AUTO_INCREMENT	PRIMARY KEY
readers	full_name	VARCHAR(100)	NOT NULL
readers	dob	DATE	cho phép NULL
readers	phone	VARCHAR(15)	cho phép NULL
loans	loan_id	INT AUTO_INCREMENT	PRIMARY KEY
loans	reader_id	INT	NOT NULL, FOREIGN KEY → readers
loans	copy_id	INT	NOT NULL, FOREIGN KEY → copies
loans	loan_date	DATE	NOT NULL
loans	status	ENUM('Pending', 'Active', 'Completed', 'Cancelled')	NOT NULL, DEFAULT 'Pending'
loans	due_date	DATE	NOT NULL, `CHECK (due_date >= loan_date)`
return_records	return_id	INT AUTO_INCREMENT	PRIMARY KEY
return_records	loan_id	INT	NOT NULL, UNIQUE, FOREIGN KEY → loans
return_records	return_date	DATE	NOT NULL
return_records	condition_note	VARCHAR(255)	cho phép NULL
return_records	fine_amount	DECIMAL(12,2)	NOT NULL, DEFAULT 0, `CHECK (fine_amount >= 0)`
payments	payment_id	INT AUTO_INCREMENT	PRIMARY KEY
payments	loan_id	INT	NOT NULL, FOREIGN KEY → loans

Bảng	Cột	Kiểu dữ liệu	Ràng buộc
payments	amount	DECIMAL(12,2)	NOT NULL, `CHECK (amount > 0)`
payments	payment_date	DATE	NOT NULL
payments	method	ENUM('Cash', 'BankTransfer', 'Card')	NOT NULL

Các lựa chọn thiết kế

- `status` và `method` dùng ENUM để chặn giá trị ngoài danh sách: đúng 4 trạng thái và 3 phương thức thanh toán của đề.
- Tiền dùng DECIMAL(12,2) để tránh sai số của kiểu số thực.
- UNIQUE trên `return_records.loan_id` biến quan hệ thành 1 – 0..1: một lượt mượn không thể có hai biên bản.
- `fine_amount` cho phép bằng 0 (trả đúng hạn, sách nguyên vẹn); `payments.amount` phải lớn hơn 0 vì mỗi dòng là một lần thu thực tế.
- Khóa ngoại dùng hành vi mặc định RESTRICT (không khai báo ON DELETE), nên không thể xóa bạn đọc hay bản sách đã có lịch sử mượn.

Ví dụ: bảng `loans`

```
CREATE TABLE loans (
    loan_id   INT  NOT NULL AUTO_INCREMENT,
    reader_id INT  NOT NULL,
    copy_id   INT  NOT NULL,
    loan_date DATE NOT NULL,
    status    ENUM('Pending', 'Active', 'Completed', 'Cancelled') NOT NULL DEFAULT
'Pending',
    due_date  DATE NOT NULL,
    PRIMARY KEY (loan_id),
    CONSTRAINT fk_loans_reader FOREIGN KEY (reader_id) REFERENCES readers (reader_id),
    CONSTRAINT fk_loans_copy   FOREIGN KEY (copy_id)   REFERENCES copies (copy_id),
    CONSTRAINT ck_loans_due_date CHECK (due_date >= loan_date)
) ENGINE = InnoDB;
```

Trigger kiểm tra quy tắc nhiều bảng

CHECK chỉ so sánh các cột trong cùng một dòng nên không diễn đạt được các quy tắc liên quan đến bảng khác (đề cũng yêu cầu không dùng CHECK để truy vấn bảng khác).

Trigger	Bảng, thời điểm	Quy tắc được kiểm tra
trg_payments_bi	payments, trước INSERT	Lượt mượn phải có biên bản với chi phí > 0; ngày thu không sớm hơn ngày lập biên bản; tổng tiền thu, kể cả khoản mới, không vượt chi phí
trg_payments_bu	payments, trước UPDATE	Như trên; tổng đã thu không tính chính dòng đang sửa
trg_return_records_bi	return_records, trước INSERT	Lượt mượn phải ở trạng thái Completed; return_date không sớm hơn loan_date

trg_return_records_bu	return_records, trước UPDATE	Như trên; chi phí không thấp hơn số tiền đã thu; ngày lập biên bản không muộn hơn ngày đã thu tiền

Mỗi trigger từ chối câu lệnh vi phạm bằng `SIGNAL SQLSTATE '45000'` kèm thông báo tiếng Việt; MySQL báo lỗi 1644. Các tình huống (i)–(o) ở mục 4 minh họa những trường hợp bị chặn.

Ví dụ: trigger `trg_payments_bi`

```
CREATE TRIGGER trg_payments_bi
BEFORE INSERT ON payments
FOR EACH ROW
BEGIN
      DECLARE v_fine        DECIMAL(12,2);
      DECLARE v_return_date DATE;
      DECLARE v_paid        DECIMAL(12,2);

      SELECT fine_amount, return_date INTO v_fine, v_return_date
      FROM return_records
      WHERE loan_id = NEW.loan_id;

      IF v_fine IS NULL THEN
      SIGNAL SQLSTATE '45000'
            SET MESSAGE_TEXT = 'Lượt mượn chưa có biên bản trả sách nên chưa thể thu
tiền';
      END IF;
      IF v_fine = 0 THEN
      SIGNAL SQLSTATE '45000'
            SET MESSAGE_TEXT = 'Lượt mượn không phát sinh chi phí nên không thu tiền';
      END IF;
      IF NEW.payment_date < v_return_date THEN
      SIGNAL SQLSTATE '45000'
            SET MESSAGE_TEXT = 'Ngày thu không được sớm hơn ngày lập biên bản';
      END IF;

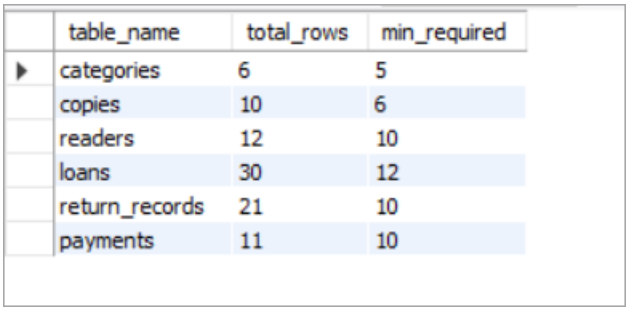
      SELECT COALESCE(SUM(amount), 0) INTO v_paid
      FROM payments
      WHERE loan_id = NEW.loan_id;

      IF v_paid + NEW.amount > v_fine THEN
      SIGNAL SQLSTATE '45000'
            SET MESSAGE_TEXT = 'Tổng tiền thu vượt chi phí đã xác nhận của lượt mượn';
      END IF;
END$$
```

Giải thích. Trigger đọc biên bản trả sách của lượt mượn (mỗi lượt có tối đa một biên bản nhờ UNIQUE nên `SELECT ... INTO` chỉ trả một dòng), rồi lần lượt loại các trường hợp chưa có biên bản, chi phí bằng 0, ngày thu quá sớm và tổng thu vượt chi phí. Trigger chạy cho từng dòng nên khoản thu thứ hai của loan 9 (20.000 + 15.000 = 35.000) và của loan 18 (11.000 + 10.000 = 21.000) vẫn được chấp nhận vì tổng không vượt chi phí.

4. Dữ liệu mẫu (seed.sql)

seed.sql nạp 6 thể loại, 10 bản sách, 12 bạn đọc, 30 lượt mượn, 21 biên bản trả và 11 lần thanh toán – đều vượt mức tối thiểu của đề – và cả 13 phép kiểm tra toàn vẹn đều trả về 0 vi phạm. Các ảnh kết quả trong báo cáo được chụp từ MySQL Workbench.



table_name	total_rows	min_required
categories	6	5
copies	10	6
readers	12	10
loans	30	12
return_records	21	10
payments	11	10

Hình 2. Số dòng từng bảng so với mức tối thiểu của đề (cột min_required)

Các tình huống đề yêu cầu và vị trí trong dữ liệu

Tình huống	Số lượng	Vị trí trong dữ liệu
Giao dịch tháng 01/2025	5 lượt	loan 4–8
Tháng 02–03/2025 (phần còn lại của quý I)	8 lượt	loan 9–16
Ngoài quý I nhưng trong năm 2025	10 lượt	loan 17–26
Ngoài năm 2025	3 lượt năm 2024, 4 lượt năm 2026	loan 1–3 và loan 27–30
Đủ 4 trạng thái	Completed 21, Cancelled 5, Pending 2, Active 2	toàn bộ 30 lượt
Bạn đọc có hơn 3 lượt Completed trong 2025	2 người	reader 1 (5 lượt), reader 2 (4 lượt); reader 3 có đúng 3 lượt nên làm ví dụ biên
Có lịch sử mượn nhưng chưa thanh toán	5 người	reader 6, 7, 8, 10, 12; reader 11 chưa mượn lần nào
Một lượt mượn trả nhiều lần	2 lượt	loan 9 và loan 18, mỗi lượt thu 2 lần
Lượt mượn còn nợ	2 lượt	loan 6 (còn 15.000), loan 25 (còn 40.000)
Thể loại chưa có giao dịch trong quý I	2 thể loại	category 5, 6
Bản sách chưa có giao dịch trong 2025	2 bản	copy 8 (chỉ mượn năm 2024 và 2026), copy 10 (chưa từng mượn)

Phần 7.5 của seed.sql đếm các tình huống trên; mọi số đếm đều từ 1 trở lên.

tinh_huong	so_luong
Lượt mượn tháng 01/2025	5
Lượt mượn tháng 02-03/2025 (còn lại của quý I)	8
Lượt mượn ngoài quý I, trong năm 2025	10
Lượt mượn trước năm 2025	3
Lượt mượn sau năm 2025	4
Trạng thái Completed	21
Trạng thái Pending	2
Trạng thái Active	2
Trạng thái Cancelled	5
Bạn đọc có hơn 3 lượt Completed trong năm 2025	2
Bạn đọc có lịch sử mượn nhưng chưa thanh toán	5
Lượt mượn được trả tiền nhiều lần	2
Lượt mượn còn nợ	2
Thể loại chưa có giao dịch trong quý I/2025	2
Bản sách chưa có giao dịch trong năm 2025	2

Hình 3. Số lượng từng tình huống trong dữ liệu mẫu

Truy vấn kiểm tra toàn vẹn

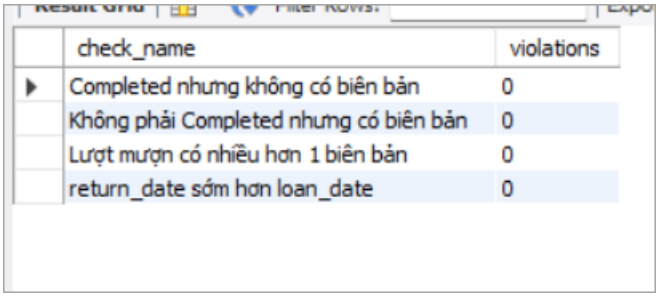
Phần 7.2 đến 7.4 của seed.sql gồm 13 phép kiểm tra chia thành ba nhóm; cả ba nhóm đều trả về 0.

Khóa ngoại (5 phép): copies → categories, loans → readers, loans → copies, return_records → loans, payments → loans.

fk_check	orphan_rows
copies.category_id -> categories	0
loans.reader_id -> readers	0
loans.copy_id -> copies	0
return_records.loan_id -> loans	0
payments.loan_id -> loans	0

Hình 4. Kiểm tra khóa ngoại: không có dòng mồ côi

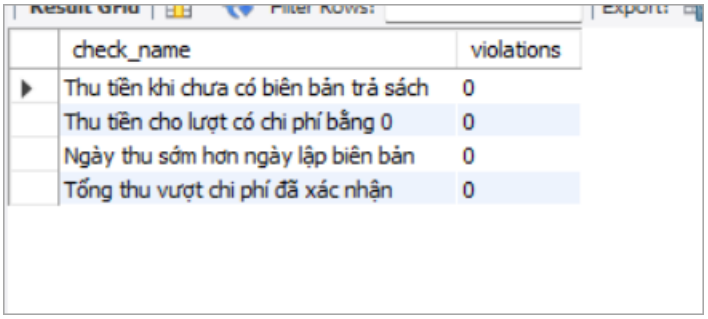
Trạng thái và biên bản (4 phép): Completed thiếu biên bản; không phải Completed nhưng có biên bản; lượt có hơn 1 biên bản; return_date sớm hơn loan_date.



check_name	violations
Completed nhưng không có biên bản	0
Không phải Completed nhưng có biên bản	0
Lượt mượn có nhiều hơn 1 biên bản	0
return_date sớm hơn loan_date	0

Hình 5. Kiểm tra trạng thái và biên bản trả sách: không có vi phạm

Thanh toán (4 phép): thu khi chưa có biên bản; thu cho lượt có chi phí bằng 0; ngày thu sớm hơn ngày lập biên bản; tổng thu vượt chi phí.



check_name	violations
Thu tiền khi chưa có biên bản trả sách	0
Thu tiền cho lượt có chi phí bằng 0	0
Ngày thu sớm hơn ngày lập biên bản	0
Tổng thu vượt chi phí đã xác nhận	0

Hình 6. Kiểm tra thanh toán: không có vi phạm

Minh họa INSERT và vi phạm ràng buộc. Mười lăm câu (a)–(o) nằm trong chú thích để tệp vẫn chạy được. Câu (a)–(h) bị ràng buộc của bảng chặn, câu (i)–(o) bị trigger chặn; cột cuối ghi kết quả mong đợi theo hành vi của MySQL:

Nhãn	Tình huống	Bị chặn bởi	Kết quả mong đợi (MySQL 8)
(a)	Trùng barcode `LIB-0001`	UNIQUE	Lỗi 1062
(b)	`category_id` = 99 không tồn tại	FOREIGN KEY	Lỗi 1452
(c)	`due_date` trước `loan_date`	CHECK	Lỗi 3819
(d)	`status` = 'Lost'	ENUM	Data truncated (strict mode)
(e)	Biên bản thứ hai cho loan 4	UNIQUE	Lỗi 1062
(f)	`fine_amount` âm (lượt Completed mới, loan 31)	CHECK	Lỗi 3819
(g)	`amount` = 0	CHECK	Lỗi 3819
(h)	`method` = 'Momo'	ENUM	Data truncated (strict mode)
(i)	Thu thêm 1.000 cho loan 9 đã thu đủ 35.000	Trigger `trg_payments_bi`	Lỗi 1644: tổng tiền thu vượt chi phí
(j)	Thu tiền cho loan 28 (Active, chưa có biên bản)	Trigger `trg_payments_bi`	Lỗi 1644: chưa có biên bản nên chưa thể thu tiền

(k)	Thu tiền cho loan 4 (chi phí bằng 0)	Trigger `trg_payments_bi`	Lỗi 1644: không phát sinh chi phí nên không thu tiền
(l)	Ngày thu 2025-02-01 sớm hơn ngày lập biên bản của loan 6 (2025-02-03)	Trigger `trg_payments_bi`	Lỗi 1644: ngày thu sớm hơn ngày lập biên bản
(m)	Lập biên bản cho loan 28 (Active)	Trigger `trg_return_records_bi`	Lỗi 1644: chỉ lập biên bản cho lượt Completed
(n)	Sửa `return_date` của biên bản 2 về trước `loan_date`	Trigger `trg_return_records_bu`	Lỗi 1644: return_date sớm hơn loan_date
(o)	Giảm chi phí biên bản 7 xuống 10.000 khi loan 9 đã thu 35.000	Trigger `trg_return_records_bu`	Lỗi 1644: chi phí thấp hơn số tiền đã thu

5. Truy vấn dữ liệu (queries.sql)

Mười truy vấn chạy độc lập và đều trả mã định danh kèm tên để phân biệt các đối tượng trùng tên. Các kỹ thuật chính là JOIN nhiều bảng, LEFT JOIN kèm điều kiện thời gian đặt trong ON để giữ dòng bằng 0, GROUP BY – HAVING và NOT EXISTS. Số tiền hiển thị theo kiểu DECIMAL(12,2), đơn vị đồng. Ảnh kết quả được chụp từ MySQL Workbench.

Câu 1. Lượt mượn Completed trong tháng 01/2025

Đề: liệt kê các lượt đăng ký mượn Completed trong tháng 01/2025, kèm bạn đọc, bản sách và ngày nghiệp vụ loan_date.

```
SELECT
    l.loan_id,
    r.reader_id,
    r.full_name AS reader_name,
    c.copy_id,
    c.barcode,
    c.title AS copy_title,
    l.loan_date,
    l.status
FROM loans l
JOIN readers r ON r.reader_id = l.reader_id
JOIN copies c  ON c.copy_id = l.copy_id
WHERE l.status = 'Completed'
  AND l.loan_date BETWEEN '2025-01-01' AND '2025-01-31'
ORDER BY l.loan_date ASC, l.loan_id ASC;
```

Giải thích. Truy vấn nối `loans` với `readers` và `copies`, lọc theo `status` và khoảng `loan_date` (ngày nghiệp vụ, không phải `return_date`). Loan 8 cũng mượn trong tháng 01/2025 nhưng đã bị hủy nên không xuất hiện.

loan_id	reader_id	reader_name	copy_id	barcode	copy_title	loan_date	status
4	1	Nguyễn Văn An	5	LIB-0005	Clean Code	2025-01-02	Completed
5	2	Trần Thị Bình	1	LIB-0001	Dế Mèn phiêu lưu ký	2025-01-08	Completed
6	3	Lê Minh Châu	3	LIB-0003	Số đỏ	2025-01-15	Completed
7	4	Phạm Quốc Dũng	6	LIB-0006	Database System Concepts	2025-01-20	Completed

Hình 7. Kết quả câu 1

Câu 2. Số lượt đăng ký mượn theo thể loại trong quý I/2025

Đề: đếm tổng số lượt đăng ký mượn theo từng thể loại sách trong quý I/2025, tính mọi trạng thái; hiển thị cả nhóm có số lượng bằng 0.

```
SELECT
    g.category_id,
    g.name AS category_name,
    COUNT(l.loan_id) AS total_loans_q1_2025
FROM categories g
LEFT JOIN copies c ON c.category_id = g.category_id
LEFT JOIN loans l  ON l.copy_id = c.copy_id
                  AND l.loan_date BETWEEN '2025-01-01' AND '2025-03-31'
GROUP BY g.category_id, g.name
ORDER BY total_loans_q1_2025 DESC, g.category_id ASC;
```

Giải thích. LEFT JOIN xuất phát từ `categories` nên thể loại không có lượt mượn vẫn hiện với số 0. Khoảng ngày nằm trong `ON` chứ không nằm trong `WHERE`: đặt ở `WHERE` sẽ loại các dòng NULL và làm mất nhóm 0. Tổng 13 lượt gồm 5 lượt tháng 1 và 8 lượt tháng 2–3.

category_id	category_name	total_loans_q1_2025
1	Văn học	5
2	Khoa học - Công nghệ	3
4	Thiếu nhi	3
3	Lịch sử	2
5	Kỹ năng sống	0
6	Ngoại ngữ	0

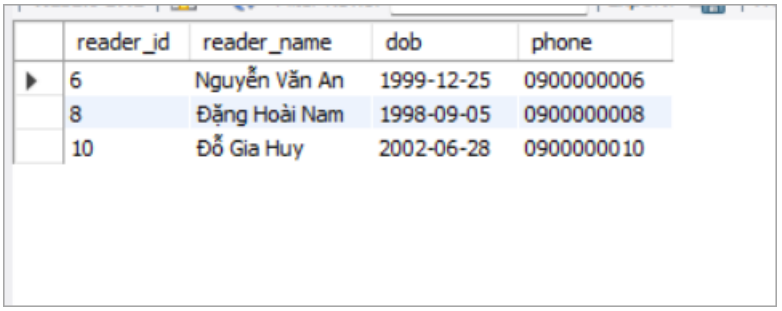
Hình 8. Kết quả câu 2

Câu 3. Bạn đọc có lượt mượn chưa có biên bản trả sách

Đề: liệt kê các bạn đọc có ít nhất một lượt đăng ký mượn chưa có biên bản trả sách, loại trừ Cancelled; mỗi người chỉ xuất hiện một lần.

```
SELECT DISTINCT
    r.reader_id,
    r.full_name AS reader_name,
    r.dob,
    r.phone
FROM readers r
JOIN loans l               ON l.reader_id = r.reader_id
LEFT JOIN return_records rr ON rr.loan_id = l.loan_id
WHERE l.status <> 'Cancelled'
  AND rr.return_id IS NULL
ORDER BY r.reader_id ASC;
```

Giải thích. LEFT JOIN `return_records` rồi lọc `rr.return_id IS NULL` để giữ lượt chưa có biên bản; `status <> 'Cancelled'` loại lượt đã hủy; DISTINCT đảm bảo mỗi người một dòng. Reader 10 có hai lượt chưa trả (loan 16 Pending, loan 29 Active) nhưng chỉ xuất hiện một lần. Reader 6 là người khác reader 1 dù cùng tên.



reader_id	reader_name	dob	phone
6	Nguyễn Văn An	1999-12-25	0900000006
8	Đặng Hoài Nam	1998-09-05	0900000008
10	Đỗ Gia Huy	2002-06-28	0900000010

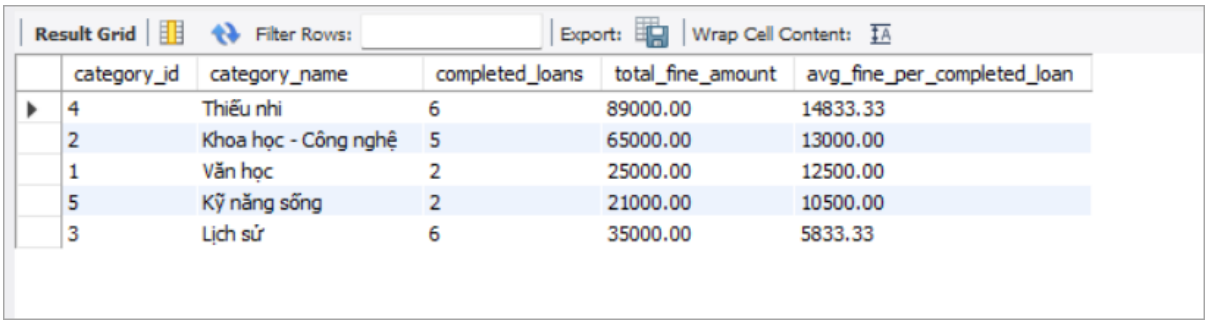
Hình 9. Kết quả câu 3

Câu 4. Phí phát sinh trung bình trên mỗi giao dịch Completed theo thể loại

Đề: tính phí phát sinh trung bình trên một giao dịch Completed của từng thể loại sách, xét toàn bộ thời gian; chỉ lấy nhóm đã phát sinh chi phí.

```
SELECT
    g.category_id,
    g.name AS category_name,
    COUNT(l.loan_id) AS completed_loans,
    SUM(rr.fine_amount) AS total_fine_amount,
    ROUND(AVG(rr.fine_amount), 2) AS avg_fine_per_completed_loan
FROM categories g
JOIN copies c          ON c.category_id = g.category_id
JOIN loans l           ON l.copy_id = c.copy_id
JOIN return_records rr ON rr.loan_id = l.loan_id
WHERE l.status = 'Completed'
GROUP BY g.category_id, g.name
HAVING SUM(rr.fine_amount) > 0
ORDER BY avg_fine_per_completed_loan DESC, g.category_id ASC;
```

Giải thích. Mỗi lượt Completed có đúng một biên bản nên AVG tính đúng trên từng giao dịch, và các phép INNER JOIN không làm nhân bản dòng. `HAVING SUM(rr.fine_amount) > 0` bỏ nhóm chưa phát sinh chi phí; với dữ liệu hiện có nó không loại nhóm nào, còn Ngoại ngữ vắng mặt vì chưa có lượt mượn.



category_id	category_name	completed_loans	total_fine_amount	avg_fine_per_completed_loan
4	Thiếu nhi	6	89000.00	14833.33
2	Khoa học - Công nghệ	5	65000.00	13000.00
1	Văn học	2	25000.00	12500.00
5	Kỹ năng sống	2	21000.00	10500.00
3	Lịch sử	6	35000.00	5833.33

Hình 10. Kết quả câu 4

Câu 5. Ba bản sách có nhiều lượt mượn Completed nhất năm 2025

Đề: liệt kê 3 bản sách có nhiều lượt đăng ký mượn Completed nhất trong năm 2025. Nếu bằng nhau, ưu tiên mã copy_id tăng dần; chỉ lấy đối tượng có giao dịch.

```
SELECT
    c.copy_id,
    c.barcode,
    c.title AS copy_title,
    c.author,
    COUNT(l.loan_id) AS completed_loans_2025
FROM copies c
JOIN loans l ON l.copy_id = c.copy_id
WHERE l.status = 'Completed'
  AND l.loan_date BETWEEN '2025-01-01' AND '2025-12-31'
GROUP BY c.copy_id, c.barcode, c.title, c.author
HAVING COUNT(l.loan_id) > 0
ORDER BY completed_loans_2025 DESC, c.copy_id ASC
LIMIT 3;
```

Giải thích. Copy 7 dẫn đầu với 4 lượt. Các copy 1, 2 và 6 cùng có 3 lượt; quy tắc phá hòa theo `copy_id` tăng dần giữ copy 1 và 2, loại copy 6. Copy 1 và 2 cùng tên "Dế Mèn phiêu lưu ký" nên phải phân biệt bằng `copy_id` và `barcode`. INNER JOIN bảo đảm chỉ có bản sách đã có giao dịch.

copy_id	barcode	copy_title	author	completed_loans_2025
7	LIB-0007	Sapiens: Lược sử loài người	Yuval Noah Harari	4
1	LIB-0001	Dế Mèn phiêu lưu ký	Tô Hoài	3
2	LIB-0002	Dế Mèn phiêu lưu ký	Tô Hoài	3

Hình 11. Kết quả câu 5

Câu 6. Tổng tiền từng bạn đọc đã thanh toán trong năm 2025

Đề: tính tổng số tiền từng bạn đọc đã thanh toán trong năm 2025 theo payment_date; hiển thị cả người chưa trả tiền với tổng bằng 0.

```
SELECT
    r.reader_id,
    r.full_name AS reader_name,
    COALESCE(SUM(p.amount), 0) AS total_paid_2025
FROM readers r
LEFT JOIN loans l    ON l.reader_id = r.reader_id
LEFT JOIN payments p ON p.loan_id = l.loan_id
                    AND p.payment_date BETWEEN '2025-01-01' AND '2025-12-31'
GROUP BY r.reader_id, r.full_name
ORDER BY total_paid_2025 DESC, r.reader_id ASC;
```

Giải thích. Bộ lọc dùng `payment_date` (ngày thu thực tế), không dùng ngày mượn: reader 3 có khoản 15.000 thu ngày 2025-01-08 cho lượt mượn năm 2024 và vẫn được tính. Lần thu thứ hai của loan 18 (10.000, ngày 2026-01-10) nằm ngoài năm 2025 nên reader 4 chỉ có 11.000. `COALESCE(SUM(...), 0)` đưa người chưa trả tiền về 0.



reader_id	reader_name	total_paid_2025
1	Nguyễn Văn An	65000.00
3	Lê Minh Châu	45000.00
2	Trần Thị Bình	25000.00
5	Hoàng Thị Hà	12000.00
9	Bùi Thu Trang	12000.00
4	Phạm Quốc Dũng	11000.00
6	Nguyễn Văn An	0.00
7	Vũ Thanh Hương	0.00
8	Đặng Hoài Nam	0.00
10	Đỗ Gia Huy	0.00
11	Ngô Khánh Linh	0.00
12	Lý Bảo Ngọc	0.00

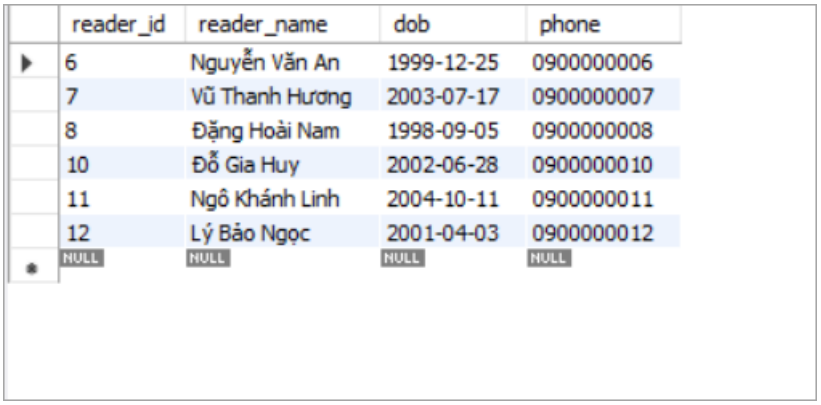
Hình 12. Kết quả câu 6

Câu 7. Bạn đọc chưa có bản ghi thanh toán nào

Đề: liệt kê các bạn đọc chưa có bất kỳ bản ghi thanh toán nào trong toàn bộ dữ liệu, kể cả người chưa có giao dịch.

```
SELECT
    r.reader_id,
    r.full_name AS reader_name,
    r.dob,
    r.phone
FROM readers r
WHERE NOT EXISTS (
    SELECT 1
    FROM loans l
    JOIN payments p ON p.loan_id = l.loan_id
    WHERE l.reader_id = r.reader_id
)
ORDER BY r.reader_id ASC;
```

Giải thích. NOT EXISTS với truy vấn con tương quan giữ lại bạn đọc không có khoản thanh toán nào trong toàn bộ dữ liệu, kể cả người chưa từng mượn như reader 11. Khác với `NOT IN`, cách này không bị ảnh hưởng bởi giá trị NULL. Reader 7 còn nợ 40.000 nhưng chưa thanh toán lần nào nên nằm trong kết quả.



reader_id	reader_name	dob	phone
6	Nguyễn Văn An	1999-12-25	0900000006
7	Vũ Thanh Hương	2003-07-17	0900000007
8	Đặng Hoài Nam	1998-09-05	0900000008
10	Đỗ Gia Huy	2002-06-28	0900000010
11	Ngô Khánh Linh	2004-10-11	0900000011
12	Lý Bảo Ngọc	2001-04-03	0900000012
NULL	NULL	NULL	NULL

Hình 13. Kết quả câu 7

Câu 8. Tổng phí phát sinh đã xác nhận của toàn bộ bạn đọc

Đề: liệt kê toàn bộ bạn đọc theo tổng phí phát sinh đã xác nhận giảm dần, không phụ thuộc đã thanh toán hay chưa; nếu bằng nhau, sắp theo mã tăng dần.

```
SELECT
    r.reader_id,
    r.full_name AS reader_name,
    COALESCE(SUM(rr.fine_amount), 0) AS total_confirmed_fine
FROM readers r
LEFT JOIN loans l           ON l.reader_id = r.reader_id
LEFT JOIN return_records rr ON rr.loan_id = l.loan_id
GROUP BY r.reader_id, r.full_name
ORDER BY total_confirmed_fine DESC, r.reader_id ASC;
```

Giải thích. Truy vấn không nối `payments` nên khoản phí không phụ thuộc đã thanh toán hay chưa và không bị cộng lặp ở lượt mượn thanh toán nhiều lần. Vì `return_records.loan_id` là UNIQUE, mỗi lượt mượn khớp tối đa một biên bản nên phép nối không nhân dòng. Reader 5 và 9 cùng 12.000 nên xếp theo mã tăng dần.



reader_id	reader_name	total_confirmed_fine
1	Nguyễn Văn An	65000.00
3	Lê Minh Châu	60000.00
7	Vũ Thanh Hương	40000.00
2	Trần Thị Bình	25000.00
4	Phạm Quốc Dũng	21000.00
5	Hoàng Thị Hà	12000.00
9	Bùi Thu Trang	12000.00
6	Nguyễn Văn An	0.00
8	Đặng Hoài Nam	0.00
10	Đỗ Gia Huy	0.00
11	Ngô Khánh Linh	0.00
12	Lý Bảo Ngọc	0.00

Hình 14. Kết quả câu 8

Câu 9. Tỷ lệ hủy của từng bản sách trong năm 2025

Đề: tính tỷ lệ hủy của từng bản sách trong năm 2025: số lượt đăng ký mượn Cancelled chia tổng số lượt đăng ký mượn của năm, nhân 100; làm tròn 2 chữ số thập phân; trả NULL nếu tổng bằng 0.

```
SELECT
    c.copy_id,
    c.barcode,
    c.title AS copy_title,
    COUNT(l.loan_id) AS total_loans_2025,
    SUM(CASE WHEN l.status = 'Cancelled' THEN 1 ELSE 0 END) AS cancelled_loans_2025,
    ROUND(
        SUM(CASE WHEN l.status = 'Cancelled' THEN 1 ELSE 0 END) * 100.0
        / NULLIF(COUNT(l.loan_id), 0),
        2
    ) AS cancel_rate_percent
FROM copies c
LEFT JOIN loans l ON l.copy_id = c.copy_id
                 AND l.loan_date BETWEEN '2025-01-01' AND '2025-12-31'
GROUP BY c.copy_id, c.barcode, c.title
ORDER BY c.copy_id ASC;
```

Giải thích. `SUM(CASE ...)` đếm lượt Cancelled; `NULLIF(COUNT(...), 0)` biến mẫu số 0 thành NULL nên tỷ lệ là NULL thay vì lỗi chia cho 0. Copy 8 và 10 không có lượt mượn nào trong năm 2025 nên có tỷ lệ NULL (copy 8 chỉ mượn năm 2024 và 2026). Copy 4 cao nhất với 2 trên 3 lượt bị hủy, tức 66.67%.

copy_id	barcode	copy_title	total_loans_2025	cancelled_loans_2025	cancel_rate_percent
1	LIB-0001	Dế Mèn phiêu lưu ký	4	1	25.00
2	LIB-0002	Dế Mèn phiêu lưu ký	3	0	0.00
3	LIB-0003	Số đỏ	2	0	0.00
4	LIB-0004	Truyện Kiều	3	2	66.67
5	LIB-0005	Clean Code	2	0	0.00
6	LIB-0006	Database System Concepts	3	0	0.00
7	LIB-0007	Sapiens: Lược sử loài người	4	0	0.00
8	LIB-0008	Việt Nam sử lược	0	0	NULL
9	LIB-0009	Đắc nhân tâm	2	1	50.00
10	LIB-0010	English Grammar in Use	0	0	NULL

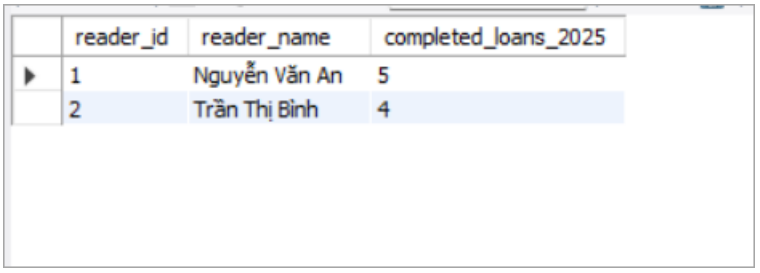
Hình 15. Kết quả câu 9

Câu 10. Bạn đọc có hơn 3 lượt mượn Completed trong năm 2025

Đề: liệt kê các bạn đọc có hơn 3 lượt đăng ký mượn Completed trong năm 2025, kèm số lần; sắp số lần giảm dần.

```
SELECT
    r.reader_id,
    r.full_name AS reader_name,
    COUNT(l.loan_id) AS completed_loans_2025
FROM readers r
JOIN loans l ON l.reader_id = r.reader_id
WHERE l.status = 'Completed'
  AND l.loan_date BETWEEN '2025-01-01' AND '2025-12-31'
GROUP BY r.reader_id, r.full_name
HAVING COUNT(l.loan_id) > 3
ORDER BY completed_loans_2025 DESC, r.reader_id ASC;
```

Giải thích. `WHERE` lọc từng dòng trước khi gom nhóm (đúng trạng thái, đúng năm), còn `HAVING` lọc nhóm sau khi đếm. "Hơn 3" là `> 3` nên reader 3 với đúng 3 lượt không đạt. Reader 1 có 5 lượt (loan 4, 9, 17, 19, 22) và reader 2 có 4 lượt (loan 5, 10, 13, 20).



reader_id	reader_name	completed_loans_2025
1	Nguyễn Văn An	5
2	Trần Thị Bình	4

Hình 16. Kết quả câu 10

6. Tạo VIEW (views.sql)

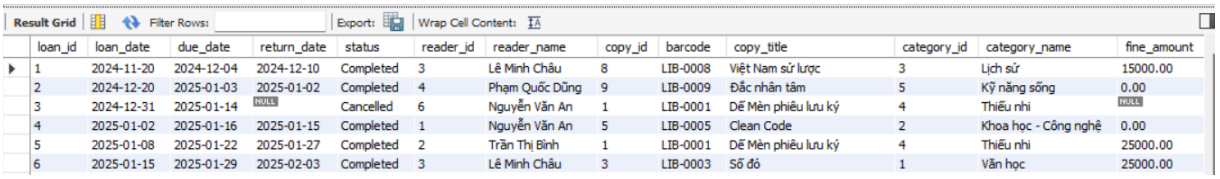
Ba VIEW xét toàn bộ thời gian: `v_detail` (chi tiết từng lượt mượn), `v_group_summary` (thống kê theo thể loại) và `v_debt` (công nợ từng bạn đọc). Điểm đáng chú ý nhất là `v_debt`, nơi tiền thanh toán phải được gộp theo lượt mượn trước khi nối để phí không bị cộng lặp.

v_detail: mỗi dòng một lượt mượn

```
CREATE VIEW v_detail AS
SELECT
    l.loan_id,
    l.loan_date,
    l.due_date,
    rr.return_date,
    l.status,
    r.reader_id,
    r.full_name AS reader_name,
    c.copy_id,
    c.barcode,
    c.title AS copy_title,
    g.category_id,
    g.name AS category_name,
    rr.fine_amount
FROM loans l
JOIN readers r              ON r.reader_id = l.reader_id
JOIN copies c               ON c.copy_id = l.copy_id
JOIN categories g           ON g.category_id = c.category_id
LEFT JOIN return_records rr ON rr.loan_id = l.loan_id;
```

Giải thích. Mỗi lượt mượn luôn có bạn đọc, bản sách và thể loại nên dùng JOIN; riêng `return_records` nối bằng LEFT JOIN để giữ các lượt Pending, Active, Cancelled với `fine_amount` là NULL. View có 30 dòng, bằng số dòng của `loans`: 21 dòng Completed có phí và 9 dòng còn lại là NULL. Bảng dưới trích 6 trong 30 dòng.

Kết quả (trích 6 trong 30 dòng)



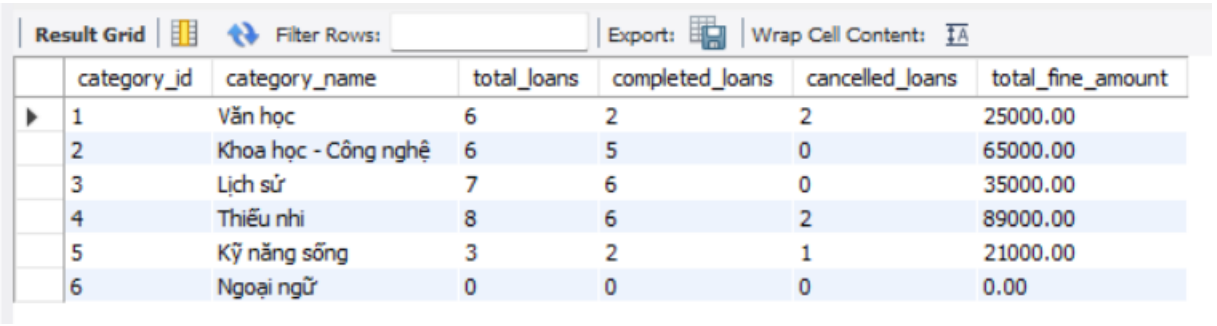
loan_id	loan_date	due_date	return_date	status	reader_id	reader_name	copy_id	barcode	copy_title	category_id	category_name	fine_amount
1	2024-11-20	2024-12-04	2024-12-10	Completed	3	Lê Minh Châu	8	LIB-0008	Việt Nam sử lược	3	Lịch sử	15000.00
2	2024-12-20	2025-01-03	2025-01-02	Completed	4	Phạm Quốc Dũng	9	LIB-0009	Đắc nhân tâm	5	Kỹ năng sống	0.00
3	2024-12-31	2025-01-14	NULL	Cancelled	6	Nguyễn Văn An	1	LIB-0001	Dế Mèn phiêu lưu ký	4	Thiếu nhi	NULL
4	2025-01-02	2025-01-16	2025-01-15	Completed	1	Nguyễn Văn An	5	LIB-0005	Clean Code	2	Khoa học - Công nghệ	0.00
5	2025-01-08	2025-01-22	2025-01-27	Completed	2	Trần Thị Bình	1	LIB-0001	Dế Mèn phiêu lưu ký	4	Thiếu nhi	25000.00
6	2025-01-15	2025-01-29	2025-02-03	Completed	3	Lê Minh Châu	3	LIB-0003	Số đỏ	1	Văn học	25000.00

v_group_summary: mỗi dòng một thể loại

```
CREATE VIEW v_group_summary AS
SELECT
    g.category_id,
    g.name AS category_name,
    COUNT(l.loan_id) AS total_loans,
    SUM(CASE WHEN l.status = 'Completed' THEN 1 ELSE 0 END) AS completed_loans,
    SUM(CASE WHEN l.status = 'Cancelled' THEN 1 ELSE 0 END) AS cancelled_loans,
    COALESCE(SUM(rr.fine_amount), 0) AS total_fine_amount
FROM categories g
LEFT JOIN copies c          ON c.category_id = g.category_id
LEFT JOIN loans l           ON l.copy_id = c.copy_id
LEFT JOIN return_records rr ON rr.loan_id = l.loan_id
GROUP BY g.category_id, g.name;
```

Giải thích. Chuỗi LEFT JOIN xuất phát từ `categories` giữ lại thể loại chưa có giao dịch (Ngoại ngữ, mọi cột bằng 0); `COUNT(l.loan_id)` không đếm dòng NULL và `COALESCE` đưa tổng phí về 0. Tổng các cột khớp dữ liệu gốc: 30 lượt, 21 Completed, 5 Cancelled và 235.000 tiền phí.

Kết quả



category_id	category_name	total_loans	completed_loans	cancelled_loans	total_fine_amount
1	Văn học	6	2	2	25000.00
2	Khoa học - Công nghệ	6	5	0	65000.00
3	Lịch sử	7	6	0	35000.00
4	Thiếu nhi	8	6	2	89000.00
5	Kỹ năng sống	3	2	1	21000.00
6	Ngoại ngữ	0	0	0	0.00

v_debt: mỗi dòng một bạn đọc

```
CREATE VIEW v_debt AS
SELECT
    r.reader_id,
    r.full_name AS reader_name,
    COALESCE(SUM(rr.fine_amount), 0) AS total_confirmed_fine,
    COALESCE(SUM(p_sum.paid_amount), 0) AS total_paid,
    COALESCE(SUM(rr.fine_amount), 0) - COALESCE(SUM(p_sum.paid_amount), 0) AS
remaining_debt
FROM readers r
LEFT JOIN loans l           ON l.reader_id = r.reader_id
LEFT JOIN return_records rr ON rr.loan_id = l.loan_id
LEFT JOIN (
    SELECT loan_id, SUM(amount) AS paid_amount
    FROM payments
    GROUP BY loan_id
) p_sum ON p_sum.loan_id = l.loan_id
GROUP BY r.reader_id, r.full_name;
```

Giải thích. Bảng con `p_sum` gộp `payments` theo `loan_id` nên mỗi lượt mượn chỉ còn một dòng khi nối. Nếu nối thẳng `payments`, phí của loan 9 và loan 18 (mỗi lượt thu hai lần) bị cộng hai lần: reader 1 sẽ bị tính 100.000 thay vì 65.000 và reader 4 là 42.000 thay vì 21.000. Công nợ bằng tổng phí đã xác nhận trừ tổng đã thu; hai người còn nợ là reader 3 (15.000, từ loan 6) và reader 7 (40.000, từ loan 25), tổng 55.000, khớp hai lượt còn nợ trong dữ liệu mẫu.

Kết quả

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

reader_id	reader_name	total_confirmed_fine	total_paid	remaining_debt
1	Nguyễn Văn An	65000.00	65000.00	0.00
2	Trần Thị Bình	25000.00	25000.00	0.00
3	Lê Minh Châu	60000.00	45000.00	15000.00
4	Phạm Quốc Dũng	21000.00	21000.00	0.00
5	Hoàng Thị Hà	12000.00	12000.00	0.00
6	Nguyễn Văn An	0.00	0.00	0.00
7	Vũ Thanh Hương	40000.00	0.00	40000.00
8	Đặng Hoài Nam	0.00	0.00	0.00
9	Bùi Thu Trang	12000.00	12000.00	0.00
10	Đỗ Gia Huy	0.00	0.00	0.00
11	Ngô Khánh Linh	0.00	0.00	0.00
12	Lý Bảo Ngọc	0.00	0.00	0.00

7. Đánh giá và hạn chế

Bài làm đáp ứng đủ các yêu cầu của đề; các lưu ý còn lại nằm ở phạm vi của trigger và môi trường chạy.

Yêu cầu của đề	Đáp ứng	Bằng chứng
Bốn tệp chạy theo thứ tự schema → seed → queries → views và chạy lại được	Có	schema.sql có DROP TABLE, views.sql có DROP VIEW IF EXISTS; mỗi tệp có `USE library_mini`
PRIMARY KEY, FOREIGN KEY, kiểu dữ liệu phù hợp, DECIMAL cho tiền	Có	Mục 3: 6 khóa chính, 5 khóa ngoại, DECIMAL(12,2)
NOT NULL, UNIQUE, CHECK, ENUM	Có	UNIQUE trên `categories.name`, `copies.barcode`, `return_records.loan_id`; 3 CHECK; 2 ENUM
Dữ liệu mẫu đủ số dòng và đủ tình huống	Có	Mục 4: 6/10/12/30/21/11 dòng và 11 tình huống
Truy vấn kiểm tra khóa ngoại, trạng thái – biên bản, thu tiền không vượt chi phí	Có	13 phép kiểm tra đều bằng 0
INSERT/UPDATE vi phạm đặt trong chú thích	Có	15 câu (a)–(o)
Chặn quy tắc nhiều bảng ngay lúc ghi dữ liệu (đề không bắt buộc)	Bổ sung	4 trigger trong schema.sql (mục 3); câu (i)–(o) ở mục 4

10 truy vấn độc lập, có mã và tên để phân biệt đối tượng trùng tên	Có	Mục 5; phân biệt reader 1 và 6, copy 1 và 2
Hiển thị nhóm bằng 0 và người chưa trả tiền với tổng 0	Có	Câu 2, 6, 8; `v_group_summary`, `v_debt`
Chi phí không bị cộng lặp khi trả nhiều lần	Có	Câu 8 không nối `payments`; `v_debt` gộp theo `loan_id`
Câu 9 trả NULL khi tổng bằng 0	Có	NULLIF; copy 8 và 10
Ba VIEW xét toàn bộ thời gian, giữ giao dịch chưa có kết quả	Có	Mục 6

Hạn chế và lưu ý

- Trigger hiện chỉ gắn vào `payments` và `return_records`. Các thay đổi trên `loans` (ví dụ đổi lượt đã có biên bản sang Cancelled, hoặc đặt `loan_date` sau `return_date`) chưa bị chặn; các truy vấn kiểm tra 7.3 vẫn phát hiện được.
- Trigger đọc tổng tiền đã thu rồi mới ghi, nên nếu hai phiên cùng thu tiền cho một lượt mượn tại cùng thời điểm thì cả hai có thể cùng vượt qua kiểm tra; muốn loại trừ cần khóa dòng (ví dụ `SELECT ... FOR UPDATE`) hoặc xử lý ở ứng dụng.
- Trigger dùng lệnh `DELIMITER`, chỉ chạy được trong MySQL Workbench hoặc `mysql` CLI; công cụ không hiểu lệnh này sẽ báo lỗi cú pháp.
- InnoDB tự tạo chỉ mục cho các cột khóa ngoại, đủ cho dữ liệu mẫu; với dữ liệu lớn có thể thêm chỉ mục trên `loans(loan_date, status)` cho các báo cáo theo ngày.

8. Kết luận

Bài thực hành hoàn thành đủ bốn phần của Đề 05: lược đồ 6 bảng có khóa chính, khóa ngoại, các ràng buộc UNIQUE, CHECK, ENUM và 4 trigger chặn các quy tắc nhiều bảng; 90 dòng dữ liệu mẫu với 13 phép kiểm tra toàn vẹn; 10 truy vấn; và 3 VIEW.

Ba bài học chính: điều kiện thời gian của bảng bên phải trong LEFT JOIN phải đặt ở ON để không mất dòng bằng 0 (câu 2, 6, 9); và khi nối chi phí với thanh toán phải gộp thanh toán theo từng lượt mượn trước để không cộng lặp (`v_debt`); và CHECK chỉ kiểm tra trong phạm vi một dòng nên quy tắc liên quan nhiều bảng cần trigger hoặc truy vấn kiểm tra.

Hướng mở rộng: thêm trigger cho thao tác sửa trên `loans`, khóa dòng khi thu tiền đồng thời và chỉ mục phục vụ các báo cáo theo ngày.